

Playwright with TypeScript Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 2.5 Months

Level: Intermediate

Course page: <https://archerinfotech.in/courses/testing-qa/playwright-training-in-pune>

Course Overview

Playwright is the automation framework built for the web as it is now — auto-waiting locators, real browser contexts, network interception and trace-based debugging, with API and UI testing in one tool. This 2.5-month programme teaches testing fundamentals, then JavaScript and TypeScript from scratch, then Playwright to production depth: fixtures, the Page Object Model, visual testing, GitHub Actions CI, and a full module on AI-assisted testing.

Why Learn Playwright with TypeScript in 2026

Playwright solved the problem that made UI automation unpopular with developers: flakiness. Its locators wait automatically for an element to be actionable rather than merely present, which removes the sleep-and-retry scaffolding that made Selenium suites brittle. Add browser contexts that isolate state without relaunching a browser, and a suite that used to take forty minutes and fail randomly runs in eight and fails only when something is genuinely broken.

For Pune specifically, the picture is honest rather than hyped. Selenium still carries more total listings, because the installed base is large and nobody rewrites a working framework for fun. But Playwright is what new projects choose, its share of Pune listings has grown every quarter, and candidates who can show a Playwright suite stand out precisely because supply has not caught up with the shift. The strongest QA profiles in 2026 show Selenium plus Playwright, not one or the other.

TypeScript is the second half of the value. Typed page objects and typed fixtures catch a renamed method at compile time instead of during a 2am CI run, and the same TypeScript carries into API testing, tooling and — if you ever want it — frontend development. This course teaches JavaScript then TypeScript from zero, so no prior web development is assumed.

- Auto-waiting locators — the end of sleep-based synchronisation
- JavaScript and TypeScript taught from zero
- API testing and UI testing in a single framework
- Network interception, mocking and visual comparison
- Trace Viewer — replay a CI failure locally instead of guessing
- GitHub Actions CI and a full AI-assisted testing module

Who This Course Is For

This course fits you if

- Manual QA testers moving into automation on a modern stack
- Selenium testers who want to add the framework new projects actually choose
- BE, BCA, BSc-CS or MCA graduates targeting QA and SDET roles
- Frontend developers taking on test ownership for their own features
- Testers who want API and UI automation in one tool rather than two

This course is not the right fit if

- Anyone wanting a pure manual-testing course — take Software Testing & QA instead
- Testers whose target employer has an established Java-and-Selenium framework they will maintain
- Anyone expecting to avoid writing code — this is a TypeScript-led track
- Engineers needing mobile-app automation, which Playwright does not cover

Detailed Curriculum

Module 1: Software Testing Fundamentals (Week 1)

The grounding every automation engineer is assumed to have and is quietly judged on in interviews. Core testing concepts, SDLC and STLC as processes rather than acronyms, and test planning as a deliverable.

Then the daily craft: writing test scenarios and test cases someone else can execute, defect management, and the distinction between functional, regression, smoke and sanity testing — including when each runs in a release cycle. Agile, Scrum and Jira close the module, because that is the process every Pune QA role operates inside.

- Software testing concepts and objectives
- SDLC and STLC
- Test planning and test strategy
- Test scenarios and test cases
- Defect management and the defect life cycle
- Functional testing
- Regression testing
- Smoke and sanity testing
- Agile and Scrum
- Jira for test management

Module 2: JavaScript Fundamentals (Weeks 1–2)

JavaScript scoped to what test automation needs, taught from the beginning. Variables and data types, operators, conditionals and loops, then arrays and objects — the structures test data lives in.

Functions and arrow functions, classes and modules follow, since page objects are classes and every framework is a set of modules. The module ends where automation actually lives: promises, `async/await` and error handling. Every Playwright call is asynchronous, so this is not background material — it is the syntax you will write on every line.

- Variables and data types
- Operators and expressions
- Conditions and control flow
- Loops and iteration
- Arrays and array methods
- Objects and destructuring
- Functions and arrow functions
- Classes and inheritance
- Modules — import and export
- Promises
- `async / await`
- Error handling with `try / catch`

Module 3: TypeScript Fundamentals (Week 2)

The type layer that makes a growing test suite maintainable. TypeScript setup and compilation, the basic types, arrays and tuples, then interfaces and type aliases — the tools you use to describe a test-data shape or an API response once and have every misuse flagged.

Functions with typed parameters and returns, classes with access modifiers for page objects, and generics for reusable helpers. The module closes on `tsconfig.json` and module resolution, because a misconfigured compiler is the most

common reason a beginner's Playwright project will not run.

- TypeScript setup and compilation
- Basic types and type inference
- Arrays and tuples
- Interfaces
- Type aliases and unions
- Typed functions and return types
- Classes and access modifiers
- Generics for reusable helpers
- Modules and module resolution
- tsconfig.json configuration

Module 4: Playwright Fundamentals (Weeks 3–4)

The core tool. Playwright's architecture and why a single WebSocket connection to a browser behaves differently from the WebDriver request cycle, project setup with the `@playwright/test` runner, and running the same suite across Chromium, Firefox and WebKit.

Pages and browser contexts are the concept to internalise: a context is an isolated profile, so tests get clean state without paying for a browser launch. Then locators — CSS, XPath, text and, preferably, role-based locators that match how a user and a screen reader find an element. Web-first assertions and auto-waiting close the module, and with them the habit of never writing a fixed sleep again.

- Introduction to Playwright and its architecture
- Project setup and the `@playwright/test` runner
- Browser automation across Chromium, Firefox and WebKit
- Pages and browser contexts for isolation
- Locators and locator strategy
- CSS and XPath locators
- Text locators
- Role-based locators and accessible naming
- Web-first assertions
- Auto-waiting and actionability checks

Module 5: Advanced Browser Automation (Weeks 4–5)

Everything a real application throws at a test. Forms, dropdowns, checkboxes and radio buttons; alerts and dialogs; frames and nested frames; multiple tabs and popups, including the pattern for capturing a window a click opens.

File uploads and downloads, mouse actions including drag and drop and hover, keyboard actions and shortcuts, and web tables with dynamic row content. The module is deliberately built around awkward pages rather than clean demo sites, because the interview question is always about the awkward one.

- Forms and input handling
- Dropdowns, checkboxes and radio buttons
- Alerts and dialogs
- Frames and nested frames
- Multiple tabs and popups
- File uploads
- File downloads
- Mouse actions, hover and drag and drop
- Keyboard actions and shortcuts
- Web tables and dynamic elements

Module 6: Playwright Test Framework (Weeks 5–6)

The runner as a framework rather than a script executor. Test structure and assertions, hooks for setup and teardown, then fixtures — Playwright's dependency-injection model — and custom fixtures, which are how you give every test an

authenticated page or a seeded API client without repeating code.

Tags and test groups for selective runs, configuration through `playwright.config.ts` including projects for browser and environment matrices, parameterised testing, parallel execution with workers, retries, and test isolation. The module ends on the trade-off between speed and independence that every suite eventually has to settle.

- Test structure and web-first assertions
- Hooks — `beforeAll`, `beforeEach`, `afterEach`, `afterAll`
- Built-in fixtures and the fixture model
- Custom fixtures for authenticated state and clients
- Tags and test groups
- `playwright.config.ts` and projects
- Parameterised testing
- Parallel execution and workers
- Retry mechanism and flake detection
- Test isolation and independence

Module 7: Page Object Model (Week 6)

Structuring a suite so that a UI change costs one edit rather than fifty. Page object design with typed locators as class members and methods that express user intent, base pages for shared behaviour, and component objects for widgets that appear on many pages — a header, a data grid, a date picker.

Utility classes, test-data separation and environment configuration complete the architecture. Folder structure is treated as a design decision with consequences, since a framework a new joiner cannot navigate is a framework that gets rewritten within a year.

- Page object design with typed locators
- Methods that express user intent
- Base pages and shared behaviour
- Component objects for reusable widgets
- Utility and helper classes
- Test data separation
- Environment configuration
- Folder structure as a design decision

Module 8: API Testing with Playwright (Week 7)

The capability that makes Playwright a single-tool framework. REST fundamentals — resources, verbs, status codes — then Playwright's request context for issuing GET, POST, PUT, PATCH and DELETE calls without a browser at all.

Headers, authentication and request payloads, then response validation and API assertions. The module's real payoff is combining the two layers: seeding state through an API call and asserting through the UI, or logging in via API to skip a slow login form in every test. That pattern alone often halves a suite's runtime.

- REST API fundamentals
- GET, POST, PUT, PATCH and DELETE
- Playwright request context
- Headers and content types
- Authentication for API calls
- Request payloads and serialisation
- Response validation and API assertions
- Combining API setup with UI assertions
- Using API login to skip slow UI flows

Module 9: Visual and Advanced Testing (Weeks 7–8)

The features that put Playwright ahead of older tools. Screenshot testing and visual comparison with baselines and tolerance thresholds, plus the discipline that stops visual tests becoming a permanent source of noise.

Network interception, request mocking and response mocking let you test error states, slow responses and edge cases that are impossible to trigger against a real backend. Authentication state and session reuse cut login cost across a whole suite. The module ends on debugging: the Trace Viewer, the Playwright Inspector, and reading a recorded trace from a CI failure instead of guessing at it.

- Screenshot testing and baselines
- Visual comparison and tolerance thresholds
- Network interception
- Request mocking
- Response mocking for error and edge states
- Saving and reusing authentication state
- Session reuse across tests
- Trace Viewer for post-mortem debugging
- Playwright Inspector and step-through debugging

Module 10: Reporting (Week 8)

Turning a run into something other people can act on. The built-in HTML reporter with its embedded traces, then Allure for the richer, historical view most Pune teams standardise on — suites, steps, attachments and trend over time.

Screenshots, videos and trace files attached on failure make a red build diagnosable without a local reproduction. The module ends on failure analysis as a practice: distinguishing a genuine defect from a flaky test from an environment problem, which is the judgement a QA engineer is actually paid for.

- The built-in HTML reporter
- Allure reports and historical trends
- Screenshots on failure
- Video recording of failing tests
- Trace files as failure artefacts
- Attaching custom data to reports
- Failure analysis — defect, flake or environment

Module 11: Git and CI/CD (Week 9)

Getting the suite running without you. Git and GitHub for automation projects — branching, pull requests and code review applied to test code, which is code and deserves the same treatment.

GitHub Actions is the primary CI target: workflow structure, the Playwright container image, browser caching, and running on push and pull request. Jenkins integration is covered for teams standardised on it. The module ends on parallel execution in CI with sharding across runners, and publishing reports as build artefacts so a failure is one click away.

- Git and GitHub for automation projects
- Branching, pull requests and review of test code
- GitHub Actions workflow structure
- The Playwright container image and browser caching
- Running on push and pull request
- Jenkins integration
- Parallel execution and sharding in CI
- Publishing reports as build artefacts

Module 12: AI-Assisted Testing (Week 10 + capstone)

The capability Pune QA panels now probe for directly, taught with the validation discipline that makes it professional.

Generative AI across the testing workflow: generating test cases from user stories, converting requirements into automated tests, and drafting Playwright scripts and TypeScript automation code.

Then the code-level applications — locator creation, page object generation, AI-assisted API testing and test-data generation — followed by the analytical ones: debugging Playwright tests with AI, analysing trace and error information, refactoring, and documentation. Test maintenance and self-healing automation are introduced as the direction the field is

moving. The module is explicit throughout that AI output is a draft, and closes on prompt engineering, coding agents, and how to validate generated tests before they enter a suite others trust.

- Generative AI for modern QA — capabilities and limits
- Generating test cases from user stories
- AI-assisted Playwright script generation
- Generating TypeScript automation code
- AI-assisted locator creation
- Converting requirements into automated tests
- Generating page object models using AI
- AI-assisted API testing
- AI-based test data generation
- Debugging Playwright tests with AI
- Analysing trace and error information
- AI-assisted code refactoring
- Generating documentation
- Prompt engineering for automation engineers
- Coding agents for test automation
- AI-assisted test maintenance
- Introduction to self-healing automation
- Validating AI-generated tests before they enter the suite

Capstone Projects

E-commerce Suite with Role-Based Locators

A complete Playwright and TypeScript suite covering an e-commerce critical path — search, product detail, cart, checkout and order confirmation — built with role-based locators, typed page objects and custom fixtures for authenticated state. Runs across Chromium, Firefox and WebKit in parallel, with the HTML reporter and traces on failure.

- Technology: Playwright
- Technology: TypeScript
- Technology: Page Object Model
- Technology: Custom fixtures
- Technology: HTML reporter
- Technology: Git

API-Seeded UI Suite with Network Mocking

A framework that demonstrates the pattern teams actually want: state seeded through Playwright's request context, authentication reused from saved storage state, and network interception used to test error states, empty states and slow responses that cannot be triggered against a live backend. Includes visual comparison on key screens.

- Technology: Playwright
- Technology: TypeScript
- Technology: Request context
- Technology: Network interception
- Technology: Storage state
- Technology: Visual comparison

Capstone — Sharded CI Suite with AI-Assisted Maintenance

The capstone runs the full suite in GitHub Actions, sharded across runners, publishing Allure reports and traces as build artefacts. It includes a documented AI-assisted workflow — tests drafted with AI from user stories, then reviewed, corrected and validated by you, with the review notes committed alongside. Pushed to GitHub with a README a reviewer can follow.

- Technology: Playwright
- Technology: TypeScript

- Technology: GitHub Actions
- Technology: Sharding
- Technology: Allure
- Technology: Trace Viewer
- Technology: AI coding assistants

Career Outcomes

Playwright sits in an unusual and favourable position in Pune hiring. Total listings are still fewer than Selenium's, because the services sector maintains large existing Java frameworks. But demand is growing every quarter, it is the default choice for new product work, and the supply of engineers who can demonstrate a real Playwright suite is thin. That mismatch is why candidates with a working Playwright portfolio often bypass the queue that Selenium freshers stand in.

The roles skew towards product companies and modern engineering teams — Cybage, BrowserStack, Druva, Helpshift, Avaamo, and the growing set of Pune startups and GCC captives building new applications. Those roles pay more than the services-sector equivalent for the same headline experience, because they expect CI ownership, API testing and some development capability. This course is built to that specification. We do not claim guaranteed placement: our institute-records placement rate is 90% across all tracks.

Roles after this course

- QA Automation Engineer
- Playwright Automation Engineer
- SDET (Software Development Engineer in Test)
- Frontend Test Engineer
- Test Automation Lead

Indicative salary bands

- QA Automation Engineer (fresher) - Rs. 4–6 LPA (source: AmbitionBox Pune QA Automation (last 12 mo))
- Automation Engineer (1–3 yrs) - Rs. 6–11 LPA (source: Indeed Pune automation testing listings (last 12 mo))
- SDET (3–6 yrs) - Rs. 10–18 LPA (source: Glassdoor Pune SDET)
- Test Architect / Sr SDET (6+ yrs) - Rs. 14–22 LPA (source: AmbitionBox Pune Test Architect)

Companies hiring for these skills

- Cybage
- BrowserStack
- Druva
- Helpshift
- Persistent Systems
- Avaamo
- Capgemini
- LTIMindtree
- Tech Mahindra
- Cognizant
- Accenture
- Saksoft
- TCS

Batch Modes and Duration

Duration: 2.5 months (about 10 weeks) for the weekday and online tracks; about 12 weeks for the weekend track

Classroom

- Location: Archer Infotech Kothrud campus (Flat No. 12, Divyadarshan Housing Society, Kothrud, Pune 411038)
- Morning batch: Monday–Friday 10:00–11:30
- Evening batch: Monday–Friday 19:00–20:30

- Saturday lab session: 10:00–13:00 (three-hour deep practice)

Live online

- Live sessions: Monday–Friday 19:30–21:00 IST
- Recordings available in the LMS within 24 hours
- Google Meet for live sessions
- GitHub for code distribution
- Batch Slack channel for doubts

Weekend

- Saturday and Sunday 10:00–13:00 (3 hours per day, 6 hours per week)
- The weekend track runs about 12 weeks instead of 10 to keep total contact hours the same

Batch sizes stay under 18 for weekday and online tracks and under 12 at weekends. New batches typically start every three to four weeks; check the live batch schedule.

Fees

The Playwright track is priced in the mid band of our catalogue, reflecting the two-and-a-half-month duration and the language coverage included before the tool itself. EMI plans are available; contact admissions for the current fee structure.

Indicative range: Rs. 28,000 – Rs. 38,000 (typical track band)

- One-time payment (5% discount)
- EMI: 50% at enrolment, 50% at four weeks
- EMI: three-month plan

Placement Support

Placement support is bundled with the course, with no separate fee. Graduates plug into the Archer placement cell built on seventeen years of relationships with Pune hiring managers. The process starts around Week 8, in parallel with your capstone, so your CV and GitHub portfolio are presentable by the time you finish.

We do not guarantee placement. Our institute-records placement rate is 90% across tracks. For Playwright specifically the positioning advice differs slightly: because listings that name Playwright are fewer, we coach graduates to apply to general automation and SDET roles and lead with the framework breadth — Playwright plus API testing plus CI — rather than filtering for the tool name.

The placement process

- Week 8: CV review, GitHub portfolio audit and positioning for automation and SDET roles
- Week 9: First mock interview (technical) with structured feedback
- Week 10: Second mock interview (HR and communication) plus an offer-negotiation primer
- Post-completion weeks 1–2: Direct introductions to partner companies hiring automation engineers
- Post-completion weeks 3–8: Weekly placement-cell check-ins and interview coaching as you apply

Partner companies

- Cybage
- Persistent Systems
- Capgemini
- LTIMindtree
- Tech Mahindra
- Amdocs
- Cognizant
- Saksoft
- TCS

Prerequisites and How to Start

The course assumes you understand what testing is for and can use a computer and command line confidently. It does not assume JavaScript, TypeScript, Playwright or any framework experience — Modules 2 and 3 exist precisely because most Playwright courses assume web knowledge that career-switching testers do not have. A typical batch mixes manual QA testers moving to automation, Selenium testers modernising, recent graduates, and a few frontend developers taking on test ownership.

None of the starting steps below are gated. Doing them beforehand means the first session is spent learning rather than installing.

Suggested first steps

- Install Node.js 20 LTS or later and confirm it with `node --version`
- Install Visual Studio Code and the official Playwright extension
- Create a free GitHub account and practise the clone, commit and push cycle once
- Run `npm init playwright@latest` in a scratch folder to see the generated project
- Skim the Playwright 'Writing tests' documentation page for about fifteen minutes for context

Frequently Asked Questions

Do I need JavaScript or TypeScript experience before joining?

No. Module 2 covers JavaScript from variables through `async/await`, and Module 3 covers TypeScript from basic types through generics. That is everything Playwright automation requires, taught before the tool appears.

Should I learn Playwright or Selenium?

Selenium has more Pune listings today; Playwright is what new projects choose and far fewer candidates can demonstrate it. If you need maximum first-job reach, start with Selenium. If you are targeting product companies, or already have Selenium and want a differentiator, choose Playwright.

Does this course cover API testing?

Yes, as a full module. Playwright has a first-class API testing layer, so you learn request context, authentication, payloads and response assertions — and the pattern of seeding state via API to make UI tests dramatically faster.

What is the Trace Viewer and why does it get its own coverage?

It records DOM snapshots, network activity, console output and every action in a run, so you can replay a CI failure locally instead of guessing. It is the main reason teams find Playwright failures easier to diagnose, and it changes how you debug.

Can Playwright test mobile applications?

No. It can emulate mobile viewports and devices in a browser, which covers responsive web testing, but it does not automate native Android or iOS apps. Appium is the tool for that and is not part of this course.

How much of the course is AI-related?

One full module of eighteen topics, plus AI-assisted work threaded through the capstone. It covers generating tests from user stories, script and page-object generation, trace analysis, self-healing concepts, and how to validate AI output before it enters a suite.

Will I finish with something I can show an interviewer?

Yes. Three projects, ending with a capstone suite on GitHub that runs sharded in GitHub Actions with Allure reports and traces published as artefacts. A working repository carries far more weight than a certificate.

What is the batch size and how often do batches start?

Under 18 for weekday and online batches, under 12 at weekends. New batches typically start every three to four weeks across weekday, weekend and live-online formats.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.